

A Hacker's Guide to SHRDLU

Copyright © 2026 Jeff Shrager (jshrager@gmail.com). All rights reserved. (See copyright details at the end of this document.)

For the curious visitor who wants to understand one of AI's most celebrated historical landmarks — without an AI PhD

Before You Arrive: What Is SHRDLU?

SHRDLU is a computer program written between 1968 and 1971 by Terry Winograd as his MIT PhD dissertation. It could hold a typed conversation in English about a simple world of colored blocks sitting on a table. You could ask it where the big green block was, tell it to pick up the red pyramid, or ask it why it had just done what it did — and it would answer sensibly, perform the action, or explain its reasoning.

For its time, this was startling. SHRDLU didn't pattern-match keywords. It genuinely *parsed* English — understood grammar, resolved pronouns, tracked context across multiple sentences. Winograd's thesis became one of the most widely read documents in early AI, and SHRDLU became a reference point in debates about machine understanding that persist to this day.

The code you're about to explore was written in a dialect of Lisp called MacLisp, running on a PDP-10 at MIT's AI Lab. It predates the Common Lisp standard by over a decade. You'll encounter idioms that look strange — `PROG` , `COND` , `THANTE` , `DEFPROP` — but the logic underneath is surprisingly clear once you know where to look.

The Map: Five Districts

SHRDLU is organized into five loosely coupled subsystems that each handle a stage of processing. A sentence enters at the left and an answer emerges at the right:

User's words

- | | |
|-------------------|--|
| → [MORPHO] | tokenize raw keystrokes into words |
| → [PROGMR/GRAMAR] | parse words into a grammatical tree |
| → [SMSPEC/SMUTIL] | translate the tree into meaning structures |
| → [PLNR/BLOCKP] | reason about the blocks world |
| → [NEWANS] | generate a natural-language reply |

Orchestrating all of this is `syscom`, which we'll visit last. Think of it as the conductor — it doesn't play an instrument, but nothing happens without it.

District 1: The Front Gate — `morpho`

Every journey through SHRDLU begins with a typed sentence. `morpho` is where raw keystrokes become words. Its central function is `ETAOIN`.

★ DON'T MISS

The name `ETAOIN`. It refers to the sequence of the most common letters in English — a mnemonic familiar to anyone who worked with Linotype typesetting machines. Burying it in a function name is a small, dry joke that's only visible if you know your typesetting history. This is the kind of thing that makes reading old code feel like archaeology.

`ETAOIN` reads characters one by one, handles special keystrokes (erase, kill-line, the escape key that puts SHRDLU into different modes), and assembles tokens into a list that the parser can work with. Words arrive exactly as typed; the lexicon handles the rest. If you type `ARE`, `morpho` passes it through unchanged — the dictionary entry for `ARE` is what links it to its base form `BE`.

There's not much to linger over here. It's a modest piece of input plumbing. But its simplicity is a deliberate choice: Winograd kept the front gate thin so all the interesting work could happen downstream.

District 2: The Parliament of Grammar — `progmr`, `ginter`, `gramar`, `cgram`, `macros`

This is one of SHRDLU's most ambitious districts, and the one most worth spending time in even if it feels bewildering at first.

SHRDLU's parser is not a standard recursive-descent parser or a context-free grammar.

Winograd invented a grammar-writing metalanguage called **PROGRAMMAR** — essentially a small programming language *for writing grammars*. Grammar rules here aren't passive patterns to match against; they're executable programs that can call semantic functions, backtrack, cut, and branch conditionally.

The files divide labor clearly: - `macros` — the *compiler* for PROGRAMMAR; transforms grammar specifications into runnable Lisp - `grammar` — the human-readable grammar source (what you'd write) - `cgram` — the compiled output of `grammar` after `macros` processes it - `progrmr` — the *interpreter* that runs the compiled grammar at parse time - `ginter` — helper functions that bridge the interpreter and the runtime state

Here's a taste of what grammar rules look like, from `grammar` — the opening of the CLAUSE rule:

```
(PDEFINE CLAUSE (POSITION-OF-PRT MVB ...)  
ENTERING-CLAUSE  
  (SETR 'TIME (BUILD TSSNODE= (MAKESYM 'TSS)) C)  
  (: (CQ SIMP) SUBJ NIL)  
  (: (CQ MAJOR) INIT SEC)  
INIT  
  (SETQ LOCATIONMARKER N)  
  (: (AND (NQ BINDER) (PARSE CLAUSE BOUND INIT)) NIL MAJOR FIXIT)  
MAJOR  
  (CUT END)  
  (COND ((EQ PUNCT '?) (GO QUEST))  
        ((OR (CQ IMPER) (EQ PUNCT '!)) (GO IMPER)))  
  ...)
```

This looks like assembly language — `GO QUEST`, `GO IMPER`, explicit labels and jumps — but it's describing the grammar of English sentences. The `:` function is a conditional branch: "if this parse succeeds, go here; if it fails, go there." `CQ` checks a feature of the current parse node. `PARSE` tries to match a syntactic category. `CUT` marks the point beyond which backtracking is forbidden.

The key insight: **grammar rules here are control flow**. When a parse attempt fails, the system automatically rewinds and tries another branch. This is what lets SHRDLU handle genuinely ambiguous sentences — it can try multiple parses and pick the one that makes semantic sense.

★ DON'T MISS

The comment buried in `grammar` near the `LOCATIONMARKER` variable:

"IF PTW HITS THE CUT POINT, THAN IT IS ASSUMED THAT SOMETHING WAS MISTAKENLY PARSED AS A MODIFIER WHEN IT WAS NOT, AND EVERYTHING IS POPPED OFF"

This is SHRDLU's whole parsing philosophy in one sentence: the grammar is *exploratory*. It tries things, checks if they made sense, and undoes them if they didn't. Parsing, for Winograd, was search — not just pattern matching.

A note on modern equivalents: If you're coming from Python or JavaScript, the closest analog to PROGRAMMAR is a parser combinator library like `pyparsing` or `parsec`. The big difference is that SHRDLU's grammar rules can reach *sideways* into the semantic layer while parsing — a grammar rule for a verb phrase can call a semantic function to check whether the action makes sense for the objects involved, before the parse even completes. Modern parsers rarely do this; the layers are kept separate. SHRDLU blurs them on purpose.

District 3: The Bazaar — `dictio`

`dictio` is the lexicon, and it's the most readable file in the whole codebase. Every word SHRDLU knows is defined here with the `DEFS` macro, giving it two things: **syntactic features** (what grammatical role it can play) and a **semantic procedure** (what to *do* with it).

```
(DEFS BALL
  SEMANTICS ((NOUN (OBJECT
                    (MARKERS: (#MANIP #ROUND)
                    PROCEDURE: ((#IS *** #BALL))))))
  FEATURES (NOUN NS))

(DEFS ABOVE
  SEMANTICS ((PREP (#LOC #ABOVE T)))
  FEATURES (PREP PLACE))

(DEFS ASK
  SEMANTICS ((VB ((TRANS (RELATION
                          (RESTRICTIONS: ((#ANIMATE)
                                              (#EVENT))
                          MARKERS: (#EVENT)
                          PROCEDURE: ((#WANT #1 #2 *TIME)))))))
  FEATURES (VB TRANS INF SUBTOB))
```

`BALL` has markers `#MANIP` (manipulable) and `#ROUND`. These markers aren't just labels — they participate in the semantic layer when the system decides whether a sentence makes

sense. You can't "put" something that isn't `#MANIP`. The grammar won't even try.

`ASK` is more elaborate: its semantic procedure generates a `RELATION` structure with explicit restrictions on what can be asking (`#ANIMATE`) and what can be asked (`#EVENT`). This is meaning encoded as procedure — the word itself carries the code that will be run to interpret it.

★ DON'T MISS

The entry for `ALL`. Its semantic value is a three-way conditional:

```
(DEFS ALL
  SEMANTICS ((DET (COND ((CQ OF) 'ALL)
                        ((MEET '(NUM DEF) FE) 'DEF)
                        ('NDET))))))
```

"All" means different things in "all of the blocks," "all five blocks," and "all blocks" — definite, quantified, and indefinite senses respectively. This single entry handles all three cases with one three-branch conditional. It's a small, elegant piece of lexical semantics that would take paragraphs to describe in a linguistics paper.

★ DON'T MISS

Irregular forms are handled with an `IRREGULAR` key rather than separate entries:

```
(DEFS AN IRREGULAR (A NIL NIL))
(DEFS ARE IRREGULAR (BE (VPL PRESENT) (INF)))
```

`AN` is just `A` in disguise. `ARE` is the plural-present form of `BE`. The lexicon is compact because irregular forms point back to their canonical entries rather than duplicating definitions. This seems obvious in retrospect, but many early NLP systems got this wrong.

District 4: The Translation Quarter — `smspec`, `smutil`, `smass`

Once the parser has built a parse tree, the semantic specialists translate it into something the planner can reason about. These three files work as a team.

`smutil` defines `BUILD` — the central constructor for two kinds of semantic objects: - **OSS**

(Object Semantic Structures): represent noun phrases, what they refer to, what kind of thing they are - **RSS** (Relation Semantic Structures): represent verb phrases and relations between things

`sm-spec` contains specialists for each grammatical construction: `SMCONJ` for conjunctions, `SMVG` for verb groups, `SMPOSS` for possessives. Each specialist knows how to take the parse tree for its construction and produce the appropriate semantic structure.

`smass` is the accessor layer — functions like `MARKERS?`, `ACTION?`, `PLAUSIBILITY?` that pull fields out of semantic structures. It's the thinnest of the three files but the most widely called; everything else uses it to read back what the specialists built.

The flow: **parse tree** → semantic specialists (`sm-spec`) → **OSS/RSS structures** (via `sm-util`) → planner queries (via `plnr`).

★ DON'T MISS

The semantic marker system is performing *selectional restriction* checks — a concept from formal linguistics. When SHRDLU encounters "pick up the idea," the `#MANIP` marker required by "pick up" is absent from "idea," and the system recognizes the sentence as semantically ill-formed *before* it tries to reason about it. The semantics layer is partly a *filter*, not just a translator. The planner never even sees sentences the semantics rejects.

District 5: The Engine Room — `plnr`

This is the heart of SHRDLU, and the most important stop on your visit.

`plnr` implements **Micro-Planner**, a theorem-proving language embedded inside Lisp. Micro-Planner was created by Gerry Sussman, Eugene Charniak, and Terry Winograd specifically for SHRDLU. It supports:

Form	What it does
<code>THASSERT</code>	Add a fact to the world model
<code>THERASE</code>	Retract a fact
<code>THGOAL</code>	Try to prove a fact (by matching or running theorems)
<code>THCONSE</code>	Consequent theorem: "if you want to prove X, here's how"
<code>THANTE</code>	Antecedent theorem: "whenever X is asserted, automatically do Y"
<code>THAND</code> / <code>THOR</code>	Logical AND and OR, with automatic backtracking
<code>THPROG</code> / <code>THFIND</code>	Procedural control within proof search

The planner maintains a global database of facts (assertions) and a stack of current goals. When SHRDLU wants to know "is there a red block on the table?", it calls `THGOAL` with that pattern. The planner searches the assertion database for a match. If it finds none, it looks for a `THCONSE` theorem that might prove it. If it finds one, it runs the theorem's body — which may assert new facts, try sub-goals, or backtrack and try something else.

Here's a glimpse of `plnr`'s read macro — the `$` prefix that makes Micro-Planner notation compact:

```
(DEFUN THREAD ; reader for the $ prefix character
  NIL
  (PROG (CHAR)
    (RETURN (COND
      ((EQ (SETQ CHAR (READCH)) '?') (LIST 'THV (READ))) ; $?X = variable l
      ((EQ CHAR '_') (LIST 'THNV (READ))) ; $_X = variable b
      ((EQ CHAR 'A) 'THASSERT) ; $A = assert sho
      ((EQ CHAR 'G) 'THGOAL) ; $G = goal short
      ...))))
```

So when you see `$?X` in a theorem, it means "the current binding of Micro-Planner variable X." When you see `$_Z`, it means "bind Z to this value." These read macros let the blocks-world theorems in `blockp` be written compactly rather than being buried in function-call syntax.

★ DON'T MISS

The very first comment in `plnr` :

```
(COMMENT DO NOT GRIND THIS FILE WITH THE STANDARD GRIND)
```

"Grind" was the Lisp pretty-printer of the era. This file's indentation and layout is so carefully hand-tuned that even the auto-formatter must not touch it. Someone — almost certainly Winograd or Sussman — cared deeply about how this code read on paper. In 1971, code was often read on paper, in printouts, and formatting was a craft.

★ DON'T MISS

The sheer count of `TH`-prefixed special forms declared at the top of `plnr`. Count them: `THAPPLY`, `THGENAME`, `THSTATE`, `THANTE`, `THERASING`, `THCONSE`, `THDUMP`, `THRESTRICT`, `THBKPT`, `THUNIQUE`, `THVSETQ`, `THMESSAGE`, `THDO`, `THGOAL`, `THERASE`, `THAND`, `THNV`, `THSUCCEED`, `THAMONG`, `THCOND`, `THSETQ`, `THASSERT`, `THASVAL`, `THERT`, `THGO`, `THFAIL`, `THOR`, `THFIND`, `THFINALIZE`, `THRETURN`, `THPROG`, `THFLUSH`, `THNOT`, `THV`. That's over thirty. Micro-Planner is not a small bolt-on — it's an entire second programming language living inside Lisp, with its own control flow, its own variable binding, and its own backtracking semantics. When someone says "SHRDLU uses Micro-Planner," they mean it uses a *language within a language*.

A note on modern equivalents: If you know Prolog, Micro-Planner will feel familiar. `THGOAL` is like a Prolog query; `THCONSE` theorems are like Prolog clauses; backtracking works the same way. The key difference is that Micro-Planner is embedded in Lisp, so theorems can call arbitrary Lisp functions mid-proof — geometry calculations, string operations, anything. Prolog keeps logic and procedures more separate. Micro-Planner deliberately blurs them, which gives it more power at the cost of predictability.

District 6: The Physical World — `blockp`, `block1`, `data`

`blockp` contains the Micro-Planner theorems specific to the blocks world. These are the rules that encode physical knowledge: what it means for something to be AT a location, to SUPPORT another object, to be CONTAINED in a box.

Here are two theorems that show the range:

```
(DEFPROP TA-AT
  (THANTE (X Y)
    (#AT $?X $?Y)
    (THRPLACA (CDR (ATAB $?X)) $?Y))
  THEOREM)
```

`TA-AT` is an *antecedent* theorem — a trigger. Whenever a fact of the form `(#AT something somewhere)` is asserted, this theorem fires automatically and updates the `ATABL` coordinate database. Moving a block in the logical world instantly updates its 3D coordinates in the geometric world.

```
(DEFPROP TA-CONTAIN
  (THANTE (X Y Z)
    (#AT $?X ?)
    (THGOAL (#MANIP $?X))
    (THGOAL (#SUPPORT $?Y $?X))
    (THOR (THAND (THGOAL (#IS $?Y #BOX)) (THVSETQ $_Z $?Y))
      (THGOAL (#CONTAIN $?Z $?Y)))
    (THASSERT (#CONTAIN $?Z $?X)))
  THEOREM)
```

`TA-CONTAIN` says: whenever a manipulable object X is placed at a new location, figure out what container (if any) it's now inside, and assert that containment relation. This is automatic world-model maintenance — physical facts cascade through the database via triggers, so the planner never has to worry about consistency.

`block1` handles the geometry: it knows the 3D coordinates of every object (stored in `ATABL`) and can answer questions like "is there room to place a 100×100×100 block at this position without colliding with anything?" These are not Micro-Planner queries — they're plain Lisp functions doing arithmetic. `block1` is where the abstract blocks world meets actual space.

`data` is the starting state of the world — the blocks as they exist when you first start talking to SHRDLU:

```
(SETQ ATABLE '(
  (:B1 (110 100 0) (100 100 100)) ; position (x y z), size (w d h)
  (:B2 (110 100 100) (100 100 100)) ; B2 sits directly atop B1
  (:B3 (400 0 0) (200 200 200)) ; a larger block, alone
  ...))
```

The coordinate space is 1000×1000 units of table surface. B2 has the same x and y as B1 but

z=100 — the height of B1 — so it's stacked on top. B3 is a larger block (200 units on each side) sitting by itself at a different location.

★ DON'T MISS

`data` also contains `DISPLAY-AS` — rendering hints for the graphics terminal:

```
(:B1 #DISPLAY #BLOCK (110 100 0) (100 100 100) RED)
(:B2 #DISPLAY #PYRAMID (110 100 100) (100 100 100) GREEN)
(:B4 #DISPLAY #PYRAMID (640 640 1) (200 200 200) BLUE)
```

B1 is a *red block*. B2 is a *green pyramid* sitting on top of it. B4 is a *blue pyramid*. SHRDLU wasn't just text — it had a live graphical display showing the blocks world update in real time as the robot arm moved. The color and shape data right here in `data` drove that display. Most people who know SHRDLU from screenshots have seen this scene; few realize that the entire rendering description is a single list literal in one source file.

District 7: The Press Office — `newans`

After the planner has figured out what's true or what action was taken, `newans` turns the result back into English. The `ANSWER` function takes the semantic interpretation of the sentence, runs the appropriate planner queries, and generates a natural-language response.

SHRDLU doesn't just say YES or NO. It can say "There are 4 of them," or "The big red one," or give a multi-sentence explanation of its reasoning. The answers are generated from the contents of the semantic structures and the results of planner queries — not pre-written canned strings.

This is a quieter part of the system architecturally, but it handles some genuinely tricky cases. Pronoun resolution, for example: SHRDLU can correctly answer "Why did you do that?" by tracing back through its own reasoning chain. `newans` reaches into the conversation history and the planner's trace to reconstruct the explanation.

District 8: The Mayor's Office — `syscom`

`syscom` is where everything comes together. The top-level function is literally called `SHRDLU` :

```

(DEFUN SHRDLU NIL
  (PROG (ERT-TIME END AMB TIMAMB BOTH BACKREF BACKREF2 ANSNAME
        LASTREL WHO PT PTW SENT PUNCT ... STATE GLOBAL-MESSAGE LEVEL
        P-TIME SMN-TIME PLNR-TIME ANS-TIME ...))
  CATCH-LOOP
  (CATCH
    (PROG NIL
      LOOP (SETQ SENTNO (ADD1 SENTNO) ...)
      UP   (SETQ N (SETQ SENT (ETAOIN)))           ; 1. read input
           (SETQ PT (SETQ C (PARSEVAL PARSEARGS))) ; 2. parse
           (SETQ INTERPRETATION (SM C))           ; 3. interpret sentence
           (TIME-ANSWER '(ANSWER C))               ; 4. generate answer
           (GO LOOP))
      ABORT-PARSER)
    (GO CATCH-LOOP)))

```

Strip away the error handling and timing instrumentation, and the core is just four operations: read, parse, interpret, answer, repeat. The `CATCH / GO CATCH-LOOP` wrapping the whole thing means that if parsing crashes for any reason, the system catches the error and loops back to the top — a primitive but effective fault tolerance mechanism. SHRDLU will never die from a bad sentence; it just says it didn't understand and asks for the next one.

★ DON'T MISS

The variable list at the top of `SHRDLU`. That `PROG` form declares over 20 local variables — the entire conversational state of the system. `BACKREF` and `BACKREF2` track pronoun referents. `LASTSENTNO` maintains discourse continuity. `GLOBAL-MESSAGE` carries error explanations. `TIMAMB` times how long it takes to process ambiguous sentences. Every conversation SHRDLU can hold fits in that list. The whole world, at that scale, was small enough to hold in your hands.

A Day in the Life: One Sentence, End to End

Let's trace the sentence "Pick up the big red block" through the entire system:

Step 1 — `morpho` / `ETAOIN` reads the keystrokes and returns the token list:

```
(PICK UP THE BIG RED BLOCK)
```

Step 2 — `progrmr` runs the compiled grammar. The `CLAUSE` rule fires, recognizes an imperative (no subject, verb first). `PICK UP` is parsed as a phrasal verb.

`THE BIG RED BLOCK` is a noun group: `THE` marks it definite, `BIG` and `RED` are adjective modifiers, `BLOCK` is the noun head. A parse tree is built.

Step 3 — `smspec` processes the parse tree. The verb group specialist sees `PICK UP`, checks its lexical entry, notes it requires `#MANIP` objects. The noun group specialist builds an OSS for "the big red block": definite, `#MANIP #RECTANGULAR` markers, with a procedure that will search the world for the matching object.

Step 4 — `plnr` executes the search and the action: - `THGOAL (#IS ?X #BLOCK)` — find all blocks → {B1, B3, B6, B7, B10} - `THGOAL (#COLOR ?X #RED)` — filter to red ones → {B1, B6} - `THGOAL (#SIZE ?X #LARGE)` — filter to large ones → {B1} (assuming B1 qualifies) - Now execute the pickup: check if the top of B1 is clear, clear it if not, then assert new `#AT` and `#GRASPING` relations - `TA-AT` fires automatically, updating B1's coordinates in `ATABLE`

Step 5 — `newans` generates the reply: `OK.`

The whole pipeline, on a 1971 PDP-10, took a few seconds.

Practical Notes for the Independent Traveler

On the Lisp dialect. This is MacLisp, not Common Lisp. Four things to know: - `DEFPROP` puts a value on a symbol's property list — in modern Lisp, `(setf (get 'SYMBOL 'THEOREM) value)` - `PROG` is roughly `let` combined with `block`, with explicit labels for `GO` jumps - `COND` is `if/elif/else` — `(COND (test1 result1) (test2 result2) (T default))` - `SETQ` is assignment

Once you internalize those four, the code opens up considerably.

On `$?X` notation. The `$` character is a read macro defined in `plnr`. `$?X` means "look up Micro-Planner variable X" — roughly equivalent to a logic variable in Prolog. `$_X` means "bind X to this value." These appear throughout `blockp` and wherever theorems are written.

On what's missing. The code in `original_repo` is not a complete running system. A 1987 email archived in `file-note` explains it plainly: Micro-Planner itself was separately maintained and is not fully present in this directory. The graphics display code is absent. Some files are ITS-era snapshots that may not be mutually consistent. What you have is the intellectual record — the algorithms and knowledge structures — not a turnkey executable.

On `commented/`. The `commented/` subdirectory contains annotated versions of several

key files added by later researchers. If a file in `original/` feels dense, check whether a commented version exists there first — those annotations can save significant time.

What Makes SHRDLU Worth Visiting

SHRDLU's fame rests on what it *could* do in 1971. But reading the code, you start to see what makes it historically interesting from a different angle: the *integration*.

Most AI systems of that era were modular experiments — a parser here, a planner there, built by different groups and rarely connected. SHRDLU wired everything together into a single coherent system where the grammar calls the planner, the planner updates the world model, and the world model constrains what the grammar will even try to parse. Semantic markers flow from lexicon entries (`dictio`) into parse decisions (`smspec`) into planner queries (`plnr`) into geometry calculations (`block1`). The marker `#MANIP` appears in the dictionary, conditions the semantics, and gates the physical theorems. It's the same concept, threaded through the entire system.

That tight coupling is also why the code can feel overwhelming at first. Nothing is cleanly separable. But that's the point — and the lesson Winograd was trying to make. Language understanding, he argued, isn't a pipeline of independent modules. It's an integrated system where syntax, semantics, and world knowledge are perpetually in conversation with each other.

Half a century later, that argument is still being made.

This guide covers the original SHRDLU source files in `original_repo/original/`. For the Micro-Planner reconstruction project, see `clplnr_repo/`.

Copyright

Hacker's Guides — guide texts, commentary, and supporting material.

Copyright © 2026 Jeff Shrager (jshrager@gmail.com). All rights reserved.

Permission requests, corrections, and questions: jshrager@gmail.com.

Scope

This copyright covers the original text of the guides and the README — the prose, structure,

commentary, and annotations written for this repository:

<https://github.com/jeffshrager/HackersGuides>

It does **not** cover:

- **The historic source code being read.** Quoted program source (and any complete source files included in this repository) remains the property of its respective authors and rights holders, or is in the public domain, as the case may be. Excerpts appear here for purposes of commentary, criticism, and scholarship. Provenance for each program's source is stated in the corresponding guide.
- **Quoted third-party material.** Brief quotations from cited historical accounts and documentation remain the property of their respective rights holders and are used for the same purposes.

Attribution

If you quote or reference a guide, please credit "Hacker's Guides, Jeff Shrager" and link to this repository: <https://github.com/jeffshrager/HackersGuides>